8.5.2 OPA/Rego

OPA/Rego demonstrates the power of integrating a general-purpose policy engine into modern infrastructure, but its flexibility comes with the trade-off of manual scoping and a steeper learning curve.

- Principal: User-defined `input.principal` object; unconstrained by the language.
- Action: String in input; unconstrained.
- Resource: Arbitrary JSON object in input; flexible but without schema enforcement.
- Context: Very strong; any data in `input` or `data` can be used in policies.
- Strengths: Very flexible, embeddable, and compatible across various infrastructure and applications.
- Limitations: Manual scoping, a steeper learning curve, and policies that can be difficult to read and maintain.

Rego's flexibility makes it invaluable in cloud-native applications—ranging from Kubernetes admission control to Terraform validation and API gateway enforcement—though it lacks a built-in structure for principals, actions, and resources.

8.5.3 OpenFGA

OpenFGA makes relationship-based authorization practical and scalable.

- Principal: First-class entities (for example, `user`) in relationship tuples; no attributes.
- Action: Named permissions in schema, derived from relationships.
- Resource: First-class typed objects with defined relations in a schema.
- No native context; must be modeled through relationships externally.
- Strengths: Perfect for ReBAC; flexible, delegable sharing; scales effectively.
- Limitations: Not well suited for fine-grained, attribute-based authorization decisions.